

Computer Architecture

CE/CS 321/330

L2

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture: 27

Cache Size

- How many total bits are required for a direct-mapped cache with 16 KiB of data and four-word blocks, assuming a 64-bit address?
- Ans
 - We know that 16 KiB is 4096 (2^{12}) words
 - With a block size of four words (2^2), there are 1024 (2^{10}) blocks (lines)
 - Each block (line) has 4×32 or 128 bits of data plus tag bits plus a valid bit
 - Tag bits = $64 - 10 - 4 = 50$ bits
 - Total number of bits =

$$2^{10} \times (128 + 50 + 1) = 179 \times 2^{10} \text{ bits} = 179 \text{ Kibibits}$$

or 22.4 KiB for a 16 KiB cache. For this cache, the total number of bits in the cache is about 1.4 times as many as needed just for the storage of the data.

Memory Hierarchy

- Four Memory Hierarchy Questions
 - Q1: Where can a block be placed in the upper level? (block placement)
 - Q2: How is a block found if it is in the upper level? (block identification)
 - Q3: Which block should be replaced on a miss? (block replacement)
 - Q4: What happens on a write? (write strategy)

Memory Hierarchy Basics

- Cache Misses (3 C's)

- Compulsory miss

- The very first access to a block cannot be in the cache, so the block must be brought into the cache. These are also called cold-start misses or first-reference misses.

- Capacity miss

- If the cache cannot contain all the blocks needed during execution of a program, capacity misses (in addition to compulsory misses) will occur because of blocks being discarded and later retrieved.
 - For set associative and direct mapped caches, if the cache were fully associative with the same total size, would this miss still occur?

- Conflict miss

- If the block placement strategy is set associative or direct mapped, conflict misses (in addition to compulsory and capacity misses) will occur because a block may be discarded and later retrieved if too many blocks map to its set. These misses are also called collision misses.

Memory Hierarchy Basics

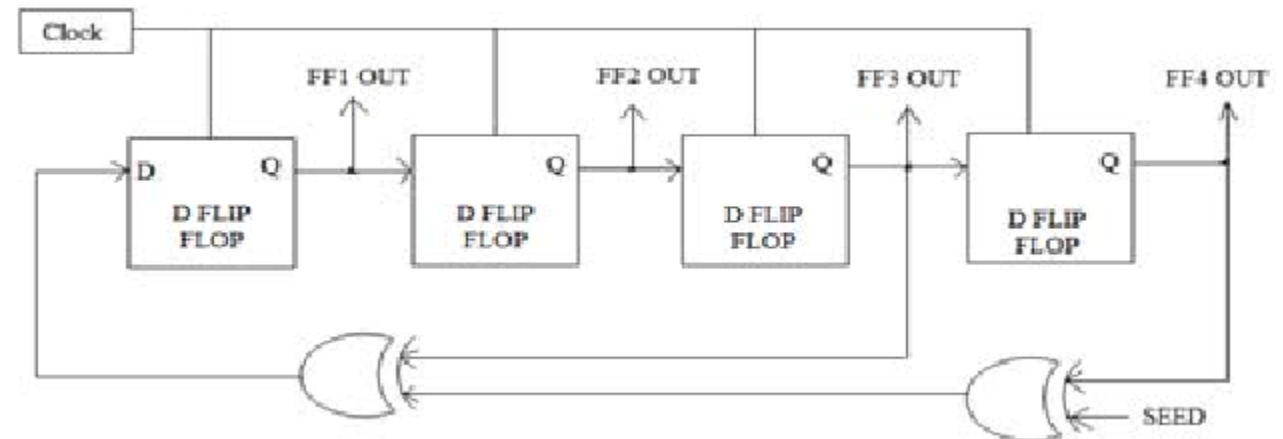
- Which block should be replaced on a Cache miss?

Block replacement

- Direct mapped
Trivial - Only one place

Replacement Algorithms for Set- and Fully-Associative caches

- Random
 - Candidate blocks are randomly selected.
 - Pseudorandom number generators (PRNG) use Linear Feedback Shift Register (LFSR) to generate a different sequence with each clock.
 - It is not exactly random since it repeats eventually after certain interval



Memory Hierarchy Basics

- Which block should be replaced on a Cache miss?

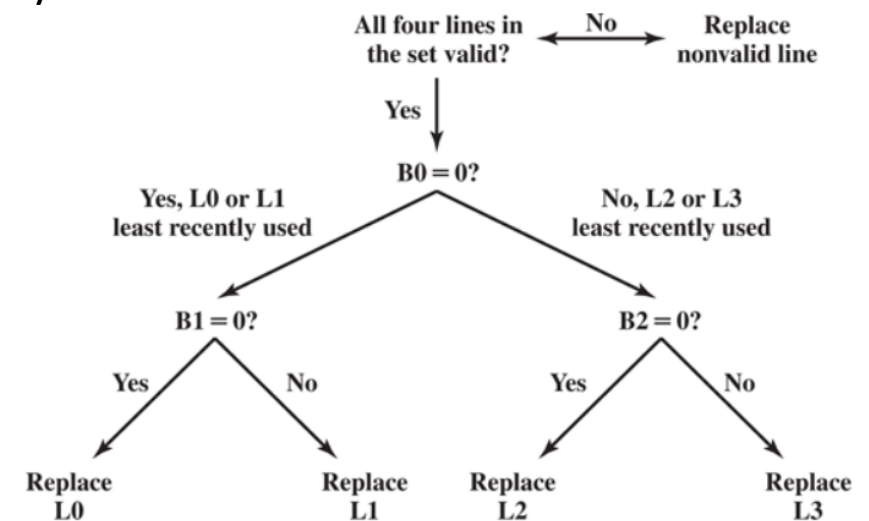
Replacement Algorithms for Set- and Fully-Associative caches

- Least Recently Used (LRU)

- Temporal locality suggests that recently used blocks are likely to be used again, then a good candidate for disposal is the least recently used block.
- The oldest unused data item, i.e. the one accessed furthest in the past, is discarded when the cache is full and a new item needs to be added
- Need to keep track of when each element in a set was used relative to the other elements in the set. Simple for 2-way, manageable for 4-way, too hard beyond that

- Pseudo LRU

- Uses a binary tree structure where internal nodes store one-bit flags, indicating the direction to the PLRU line.
- For 4-way three bits are needed for each half, B1 for L0 and L1, B2 for L2 and L3 and B0 for selecting between the two halves (L0L1 and L2L3)



Memory Hierarchy Basics

- Which block should be replaced on a Cache miss?

Replacement Algorithms for Set- and Fully-Associative caches

- Most Recently Used:
 - Unlike LRU, MRU discards the most-recently-used items first. MRU algorithms are most useful in situations where the older an item is, the more likely it is to be accessed.
- Simple Queue Based Policies
 - FIFO: With this algorithm, the cache behaves like a FIFO queue; it evicts blocks in the order in which they were added, regardless of how often or how many times they were accessed before.
 - LIFO: The cache behaves like a stack. The cache evicts the block added most recently first, regardless of how often or how many times it was accessed before.
- Least Frequently Used
 - The LFU algorithm counts how often an item is accessed in the past; those used less often are discarded first.

Example

- Assume a 2-way set associative cache having six cache lines and one-word per line
- Consider the following word number sequence as requested by the processor
0, 1, 6, 4, 2, 3, 4, 0, 6, 4, 5, 3, 7, 0, 1, 0, 3, 6, 7, 5
- Assuming an LRU replacement scheme, show which accesses are hit or miss. For miss also determine the type of miss
 - For this cache there are three sets with two lines per set. Since line (block) size is equal to a word size, the word number could be used to find the set number to which it will go using the formula
$$\text{Set Number} = (\text{word number}) \bmod (\# \text{ sets})$$
- Belady's algorithm (optimal replacement algorithm)
 - The most efficient cache block replacement algorithm would be to discard information which would not be needed for the longest time
 - This is obviously unfeasible in practice as it is impossible to predict how far in the future information will be needed.

Belady's algorithm

Which block should be replaced?

We have reached the capacity

11 misses

Miss rate = $\frac{11}{20}$
= 0.55

Word Number	Cache Index	Hit/Miss	Type	Cache Index					
				Set 0		Set 1		Set 2	
				Block 0	Block 1	Block 0	Block 1	Block 0	Block 1
0	0	M	Comp	M[0]					
1	1	M	Comp	M[0]		M[1]			
6	0	M	Comp	M[0]	M[6]	M[1]			
4	1	M	Comp	M[0]	M[6]	M[1]	M[4]		
2	2	M	Comp	M[0]	M[6]	M[1]	M[4]	M[2]	
3	0	M	Comp	M[0]	M[3]	M[1]	M[4]	M[2]	
4	1	H		M[0]	M[3]	M[1]	M[4]	M[2]	
0	0	H		M[0]	M[3]	M[1]	M[4]	M[2]	
6	0	M	Conf	M[6]	M[3]	M[1]	M[4]	M[2]	
4	1	H		M[6]	M[3]	M[1]	M[4]	M[2]	
5	2	M	Comp	M[6]	M[3]	M[1]	M[4]	M[2]	M[5]
3	0	H		M[6]	M[3]	M[1]	M[4]	M[2]	M[5]
7	1	M	Comp	M[6]	M[3]	M[1]	M[7]	M[2]	M[5]
0	0	M	Cap	M[0]	M[3]	M[1]	M[7]	M[2]	M[5]
1	1	H		M[0]	M[3]	M[1]	M[7]	M[2]	M[5]
0	0	H		M[0]	M[3]	M[1]	M[7]	M[2]	M[5]
3	0	H		M[0]	M[3]	M[1]	M[7]	M[2]	M[5]
6	0	M	Cap	M[0]	M[6]	M[1]	M[7]	M[2]	M[5]
7	1	H		M[0]	M[6]	M[1]	M[7]	M[2]	M[5]
5	2	H		M[0]	M[6]	M[1]	M[7]	M[2]	M[5]

LRU algorithm

'*' - indicates the LRU block

We have reached the capacity

14 misses

Miss rate = $14/20$
= 0.7

Word Number	Cache Index	Hit/Miss	Type	Cache Index					
				Set 0		Set 1		Set 2	
				Block 0	Block 1	Block 0	Block 1	Block 0	Block 1
0	0	M	Comp	M[0]					
1	1	M	Comp	M[0]		M[1]			
6	0	M	Comp	M[0]*	M[6]	M[1]			
4	1	M	Comp	M[0]*	M[6]	M[1]*	M[4]		
2	2	M	Comp	M[0]*	M[6]	M[1]*	M[4]	M[2]	
3	0	M	Comp	M[3]	M[6]*	M[1]*	M[4]	M[2]	
4	1	H		M[3]	M[6]*	M[1]*	M[4]	M[2]	
0	0	M	Conf	M[3]*	M[0]	M[1]*	M[4]	M[2]	
6	0	M	Conf	M[6]	M[0]*	M[1]*	M[4]	M[2]	
4	1	H		M[6]	M[0]*	M[1]*	M[4]	M[2]	
5	2	M	Comp	M[6]	M[0]*	M[1]*	M[4]	M[2]*	M[5]
3	0	M	Cap	M[6]*	M[3]	M[1]*	M[4]	M[2]*	M[5]
7	1	M	Comp	M[6]*	M[3]	M[7]	M[4]*	M[2]*	M[5]
0	0	M	Cap	M[0]	M[3]*	M[7]	M[4]*	M[2]*	M[5]
1	1	M	Cap	M[0]	M[3]*	M[7]*	M[1]	M[2]*	M[5]
0	0	H		M[0]	M[3]*	M[7]*	M[1]	M[2]*	M[5]
3	0	H		M[0]*	M[3]	M[7]*	M[1]	M[2]*	M[5]
6	0	M	Cap	M[6]	M[3]*	M[7]*	M[1]	M[2]*	M[5]
7	1	H		M[6]	M[3]*	M[7]	M[1]*	M[2]*	M[5]
5	2	H		M[0]*	M[6]	M[7]	M[1]*	M[2]*	M[5]

Memory Hierarchy

- Four Memory Hierarchy Questions
 - Q1: Where can a block be placed in the upper level? (block placement)
 - Q2: How is a block found if it is in the upper level? (block identification)
 - Q3: Which block should be replaced on a miss? (block replacement)
 - Q4: What happens on a write? (write strategy)

Cache Write Policy

- What happens on a write?

There are two basic options when writing to the cache:

- Write-through

- The information is written to both the block in the cache and to the block in the lower-level memory.

- Write-back

- The information is written only to the block in the cache. The modified cache block is written to main memory only when it is replaced.
- To mark that a block has been changed by the processor after the write operation, each cache line has an associated dirty (modified) bit as well.
- When the block is selected for replacement we check the dirty bit, if it has been modified then we copy the block otherwise we simply overwrite it.

Computer Architecture

CE/CS 321/330

L2

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture: 28

Measuring Cache Performance

- A simple formula for finding the CPU execution time was

$$CPU\ time = CPU\ clock\ cycles\ for\ a\ program \times Clock\ cycle\ time$$

$$CPU\ Clock\ cycles\ for\ a\ program = Instruction\ count \times CPI$$

- With a memory system involving caches, CPU clock cycles can be divided into
 - the clock cycles that the CPU spends executing the program and
 - the clock cycles that the CPU spends waiting for the memory system

$$CPU\ time = (CPU\ execution\ clock\ cycles + Memory\text{-}stall\ clock\ cycles) \times Clock\ cycle\ time$$

- The memory-stall clock cycles come primarily from cache misses

Measuring Cache Performance

$CPU\ time = (CPU\ execution\ clock\ cycles + Memory\text{-}stall\ clock\ cycles) \times Clock\ cycle\ time$

- If reads and writes have different miss penalties then, memory-stall clock cycles can be defined as the sum of the stall cycles coming from reads plus those coming from writes

$Memory\text{-}stall\ clock\ cycles = (Read\text{-}stall\ cycles + Write\text{-}stall\ cycles)$

- The read-stall cycles can be defined in terms of the number of read accesses per program, the miss penalty in clock cycles for a read, and the read miss rate

$Read\text{-}stall\ cycles = \frac{Reads}{Program} \times Read\ miss\ rate \times Read\ miss\ penalty$

- And similarly for write miss

$Write\text{-}stall\ cycles = \frac{Writes}{Program} \times Write\ miss\ rate \times Write\ miss\ penalty$

Measuring Cache Performance

$$CPU\ time = (CPU\ execution\ clock\ cycles + Memory\text{-}stall\ clock\ cycles) \times Clock\ cycle\ time$$

- For simplicity we can assume the reads and writes miss-penalties and miss rates to be same, therefore, we can combine the reads and writes by using a single miss rate and miss penalty.

$$Memory\text{-}stall\ clock\ cycles = \frac{Memory\ accesses}{Program} \times Miss\ rate \times Miss\ penalty$$

Measuring Cache Performance

$CPU\ time = (CPU\ execution\ clock\ cycles + Memory\text{-}stall\ clock\ cycles) \times Clock\ cycle\ time$

- Similarly, instruction cache and data cache can have different miss rates and so memory-stall clock cycles can also be defined in terms of instruction and data.

$Memory\text{-}stall\ clock\ cycles = (Instruction\ cache\ stall\ cycles + Data\ cache\ stall\ cycles)$

- Stalls due to Instruction accesses

$Instruction\ cache\text{-}stall\ cycles = \frac{Instructions}{Program} \times Instruction\ cache\ miss\ rate \times miss\ penalty$

- And similarly, stalls due to data accesses

$Data\ cache\text{-}stall\ cycles = \frac{Data\ accesses}{Program} \times Data\ cache\ miss\ rate \times miss\ penalty$

Data accesses are through the load and store instructions

Example

- Assume the miss rate of an instruction cache is 2% and the miss rate of the data cache is 4%. If a processor has a CPI of 2 without any memory stalls, and the miss penalty is 100 cycles for all misses, determine how much faster a processor would run with a perfect cache that never missed. Assume the frequency of all loads and stores is 36%.

CPU time = (CPU execution clock cycles + Memory-stall clock cycles) $\times$ Clock cycle time

CPU execution clock cycle = Instruction count $\times$ CPI

Memory-stall clock cycles = $\frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$

— For a perfect cache

- There are no memory stall clock cycles
- CPU time with perfect cache = IC $\times$ CPI $\times$ clock cycle time
= I $\times$ 2.0 $\times$ T

Example

- Assume the miss rate of an instruction cache is 2% and the miss rate of the data cache is 4%. If a processor has a CPI of 2 without any memory stalls, and the miss penalty is 100 cycles for all misses, determine how much faster a processor would run with a perfect cache that never missed. Assume the frequency of all loads and stores is 36%.

$$\text{CPU time} = (\text{CPU execution clock cycles} + \text{Memory-stall clock cycles}) \times \text{Clock cycle time}$$

$$\text{CPU execution clock cycle} = \text{Instruction count} \times \text{CPI}$$

$$\text{Memory-stall clock cycles} = \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$$

– For memory with stalls

- Memory-stall clock cycles = Instruction cache stall cycles + Data cache stall cycles
- Instruction cache stall cycles = Instruction accesses/program $\times$ I-cache miss rate $\times$ miss penalty
$$= I \times 0.02 \times 100$$
$$= I \times 2.0$$
- Data cache stall cycles = Data accesses/program $\times$ D-cache miss rate $\times$ miss penalty
$$= (I \times 0.36) \times 0.04 \times 100$$
$$= I \times 1.44$$

Example

- Assume the miss rate of an instruction cache is 2% and the miss rate of the data cache is 4%. If a processor has a CPI of 2 without any memory stalls, and the miss penalty is 100 cycles for all misses, determine how much faster a processor would run with a perfect cache that never missed. Assume the frequency of all loads and stores is 36%.

$$\text{CPU time} = (\text{CPU execution clock cycles} + \text{Memory-stall clock cycles}) \times \text{Clock cycle time}$$

$$\text{CPU execution clock cycle} = \text{Instruction count} \times \text{CPI}$$

$$\text{Memory-stall clock cycles} = \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$$

– For memory with stalls

- Memory-stall clock cycles = Instruction cache stall cycles + Data cache stall cycles
$$= I \times 2.0 + I \times 1.44 = I \times 3.44$$
- CPU time with stalls = (IC x CPI + memory stalls) x clock cycle time
$$= (I \times 2.0 + I \times 3.44) \times T = I \times 5.44 \times T$$

This is more than three cycles of memory stall per instruction

Example

- Assume the miss rate of an instruction cache is 2% and the miss rate of the data cache is 4%. If a processor has a CPI of 2 without any memory stalls, and the miss penalty is 100 cycles for all misses, determine how much faster a processor would run with a perfect cache that never missed. Assume the frequency of all loads and stores is 36%.

CPU time = (CPU execution clock cycles + Memory-stall clock cycles) × Clock cycle time

CPU execution clock cycle = Instruction count × CPI

Memory-stall clock cycles = $\frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$

– Speed up

$$\frac{\text{CPU time with stalls}}{\text{CPU time with perfect cache}} = \frac{I \times 5.44 \times T}{I \times 2.0 \times T} = \frac{5.44}{2} = 2.72$$

The perfect cache system is 2.72 times faster

– The amount of execution time spent on memory stalls

$$3.44/5.44 = 63\%$$

Cache Optimizations

Average memory access time (AMAT) = Hit time + Miss rate $\times$ Miss penalty

- How can we improve AMAT?
 - Any of the three parameters could be optimized, individually or collectively
- Basic Cache Optimizations
 - Reducing the miss rate
 - larger block size
 - larger cache size
 - higher associativity
 - Reducing the miss penalty
 - multilevel caches
 - Reducing the time to hit in the cache
 - Small and simple first level cache

Cache Optimizations

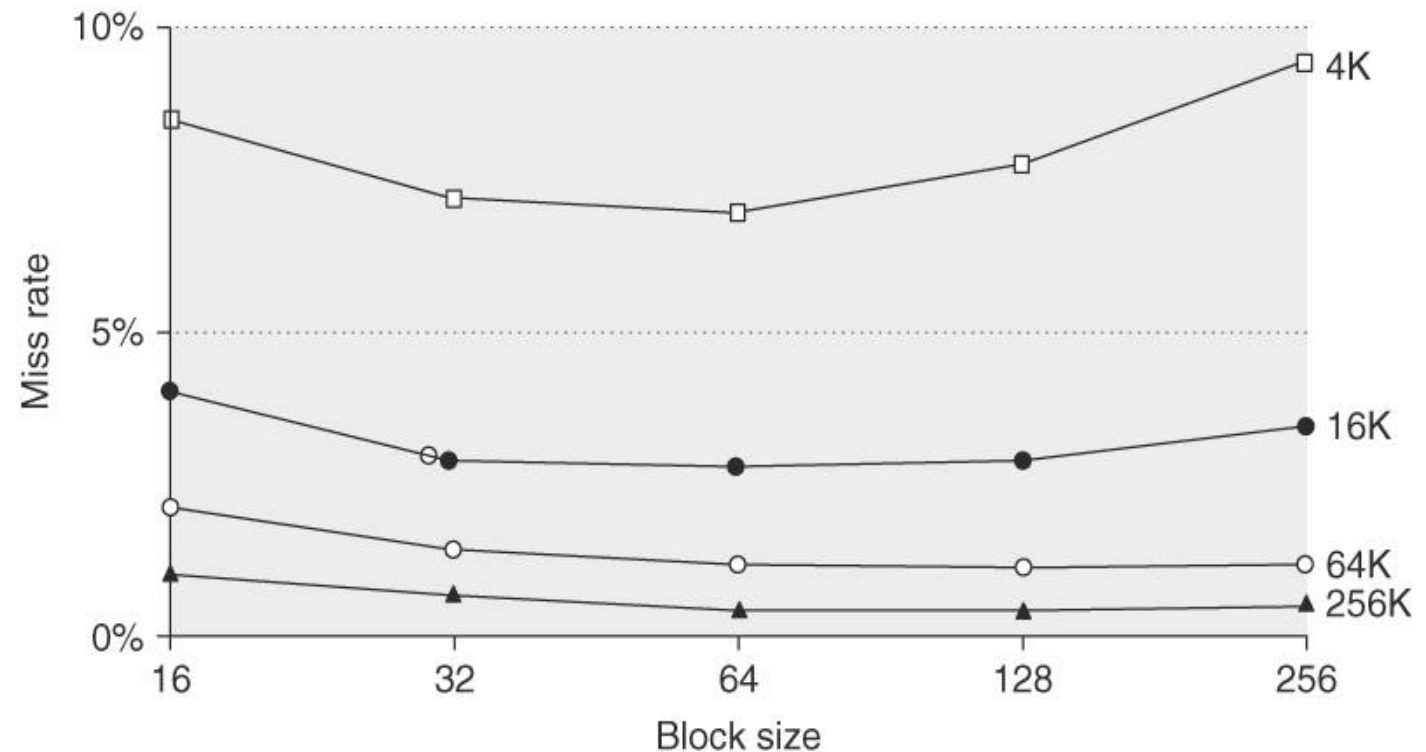
- Larger Block Size to Reduce Miss Rate

- The simplest way to reduce miss rate is to increase the block size. Larger blocks take advantage of spatial locality.

- Larger block sizes will reduce compulsory misses.

- However,

- Larger blocks increase the miss penalty (miss latency).
- Since they reduce the number of blocks in the cache, larger blocks may increase conflict misses and even capacity misses if the cache is small.



Example

- Cache block size (B) can affect both miss rate and miss penalty. Assuming a machine with a base CPI of 1, and an average of 1.35 references (both instruction and data) per instruction, find the block size that minimizes the total miss latency given the following miss rates for various block sizes. Cache block size is assumed to be in bytes.

Block size : miss rate	8 : 4%	16 : 3%	32 : 2%	64 : 1.5%	128 : 1%
------------------------	--------	---------	---------	-----------	----------

- a) What is the optimal block size for a miss penalty of $20 \times B$ cycles?
b) What is the optimal block size for a miss penalty of $24 + B$ cycles?

a) Total miss latency = miss rate x miss penalty

$$\text{for } B = 8: 0.040 \times (20 \times 8) = 6.40$$

$$\text{for } B = 16: 0.030 \times (20 \times 16) = 9.60$$

$$\text{for } B = 32: 0.020 \times (20 \times 32) = 12.80$$

$$\text{for } B = 64: 0.015 \times (20 \times 64) = 19.20$$

$$\text{for } B = 128: 0.010 \times (20 \times 128) = 25.60$$

B = 8 is the best

b) Total miss latency = miss rate x miss penalty

$$\text{for } B = 8: 0.040 \times (24 + 8) = 1.28$$

$$\text{for } B = 16: 0.030 \times (24 + 16) = 1.20$$

$$\text{for } B = 32: 0.020 \times (24 + 32) = 1.12$$

$$\text{for } B = 64: 0.015 \times (24 + 64) = 1.32$$

$$\text{for } B = 128: 0.010 \times (24 + 128) = 1.52$$

B = 32 is the best

Cache Optimizations

- Larger Caches to Reduce Miss Rate
 - The obvious way to reduce capacity misses is to increase capacity of the cache.
 - The drawback is potentially longer hit time and higher cost and power.
 - This technique has been especially popular in off-chip caches.

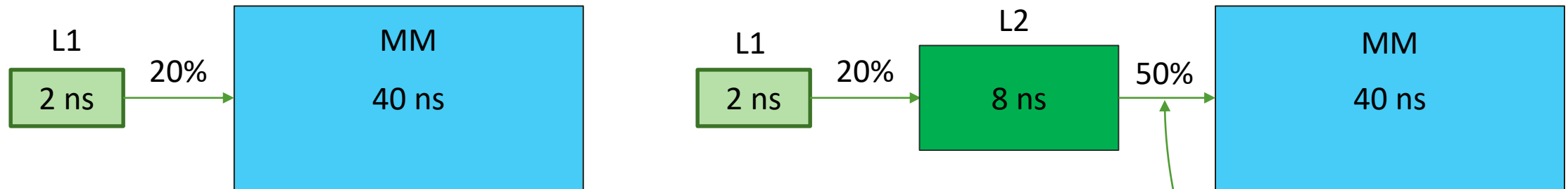
Cache Optimizations

- Higher Associativity to Reduce Miss Rate
 - **2:1 cache rule of thumb**, is that a direct mapped cache of size N has about the same miss rate as a two-way set associative cache of size $N/2$.
 - Higher associativity will reduce conflict misses.
 - The drawback is potentially longer hit time

Cache Optimizations

- Multilevel Caches to Reduce Miss Penalty

- Adding another level of cache between the original cache and memory improves the miss penalty.
 - The first-level cache can be small enough to match the clock cycle time of the fast processor.
 - The second-level cache can be large enough to capture many accesses that would go to main memory, thereby lessening the effective miss penalty.



$$\begin{aligned}\text{Average memory access time} &= \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times \text{Miss penalty}_{L1} \\ &= 2 \text{ ns} + 0.2 \times 40 \text{ ns} = 10 \text{ ns}\end{aligned}$$

Local miss rate

Miss rates in multilevel caches

- Local miss rate

- ratio of all misses in the secondary cache divided by the number of accesses to it.
- Cache misses relative to its own accesses.

- L1 local miss rate = $\frac{L1 \text{ misses}}{L1 \text{ accesses}}$ e.g. $20 / 100 = 0.2 = 20\%$

- L2 local miss rate = $\frac{L2 \text{ misses}}{L1 \text{ misses}} = \frac{L2 \text{ misses}}{L2 \text{ accesses}}$ e.g. $8 / 20 = 0.4 = 40\%$

- Global miss rate

- Global miss rate measures how many of the original memory references (from CPU) miss all cache levels up to that level.

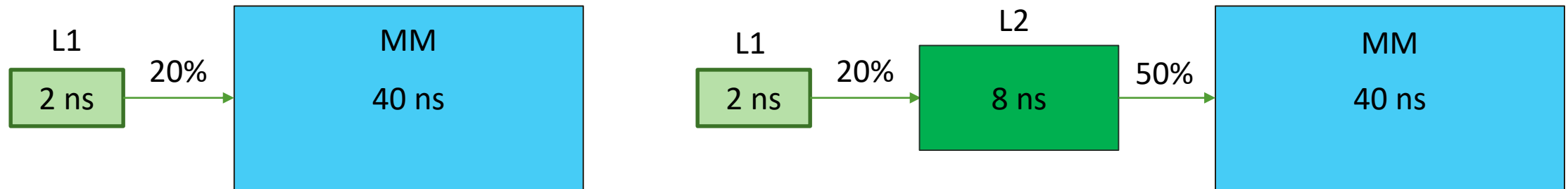
- L2 global miss rate = $\frac{L2 \text{ misses}}{L1 \text{ accesses}}$
= L1 local miss rate $\times$ L2 local miss rate
= $0.2 \times 0.4 = 0.08 = 8\%$

global miss rate of L2 is never greater than the local miss rate of L1

Cache Optimizations

- Multilevel Caches to Reduce Miss Penalty

- Adding another level of cache between the original cache and memory improves the miss penalty.
 - The first-level cache can be small enough to match the clock cycle time of the fast processor.
 - The second-level cache can be large enough to capture many accesses that would go to main memory, thereby lessening the effective miss penalty.



$$\begin{aligned}\text{Average memory access time} &= \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times \text{Miss penalty}_{L1} \\ &= 2 \text{ ns} + 0.2 \times 40 \text{ ns} = 10 \text{ ns}\end{aligned}$$

$$\begin{aligned}\text{Average memory access time} &= \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}) \\ &= 2 \text{ ns} + 0.2 \times (8 \text{ ns} + 0.5 \times 40 \text{ ns}) = 7.6 \text{ ns}\end{aligned}$$